Lab 6: Code and Branch Coverage Analysis

Name: Vasu Tripathi
SRN: PES1UG24AM918
Section: G
Repository: https://github.com/Vasu-Tripathi/L6_L7_39_password_policy

---

Objective

Measure and improve line (code) coverage and branch coverage of password_policy.py using the coverage and pytest tools, writing test cases until each metric reaches at least 90%.

Module Under Test

password_policy.py contains three functions:

- `check_password(pw)` – returns a list of rule violations (length, common password, missing character classes).
- `strength(pw)` – rates a password as strong, good, weak or very weak.
- `suggest_fix(issues)` – suggests a fix for the first reported issue.

The starter test file (two tests) achieved only 55% line coverage and 52% branch coverage.

1. Code (Line) Coverage

Test file: `CodeCoverageTest_password_policy.py` (15 tests)

```
$ python -m coverage run -m pytest CodeCoverageTest_password_policy.py
$ python -m coverage report -m
```

```
vasut@Vasu MINGW64 ~/Documents/SE LAB 6/L6_L7_39_password_policy/lab6 (main)
$ python -m pip install coverage pytest hypothesis
python -m coverage run -m pytest CodeCoverageTest_password_policy.py
python -m coverage report -m
Requirement already satisfied: colorama>=0.4 in C:\Program Files\Python311\Lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=2 in C:\Users\vasut\AppData\Roaming\Python\Python311\site-packages (from pytest) (2.3.1)
Requirement already satisfied: packaging>=24 in C:\Program Files\Python311\Lib\site-packages (from pytest) (26.3)
Requirement already satisfied: pluggy<2,>=1.6 in C:\Users\vasut\AppData\Roaming\Python\Python311\site-packages (from pytest) (1.7.0)
Requirement already satisfied: pygments>=2.15 in C:\Program Files\Python311\Lib\site-packages (from pytest) (2.21.0)
Requirement already satisfied: sortedcontainers<3.0.0,>=2.1.0 in C:\Users\vasut\AppData\Roaming\Python\Python311\site-packages (from h

[notice] A new release of pip is available: 26.2.1 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
============================================================ test session starts ============================================
platform win32 -- Python 3.11.0, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\vasut\Documents\SE LAB 6\L6_L7_39_password_policy\lab6
plugins: hypothesis-6.170.15, platformdirs-4.12.2, anyio-4.14.2, typeguard-4.6.0
collected 15 items

CodeCoverageTest_password_policy.py ...............

============================================================ warnings summary ===============================================
..\..\..\..\..\..\Program Files\Python311\Lib\site-packages\anyio\pytest_plugin.py:14
  C:\Program Files\Python311\Lib\site-packages\anyio\pytest_plugin.py:14: PytestRemovedIn10Warning: _pytest.python.CallSpec2 has been
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================================== 15 passed, 1 warning in 1.29s ========================================
Name                                Stmts   Miss  Cover   Missing
-----------------------------------------------------------------
CodeCoverageTest_password_policy.py      33      0   100%
password_policy.py                       47      0   100%
-----------------------------------------------------------------
TOTAL                                    80      0   100%
```

Figure 1: Line coverage – 15 passed, password_policy.py 100% (47/47 statements)

Result: all 47 statements of password_policy.py executed; 0 missing lines. The tests cover each rule in check_password (too short, too long, common, each missing character class), every strength rating, and every return path of suggest_fix.

2. Branch Coverage

Test file: `BranchCoverageTest_password_policy.py` (26 tests)

```
$ python -m coverage run --branch -m pytest
BranchCoverageTest_password_policy.py

$ python -m coverage report -m
```

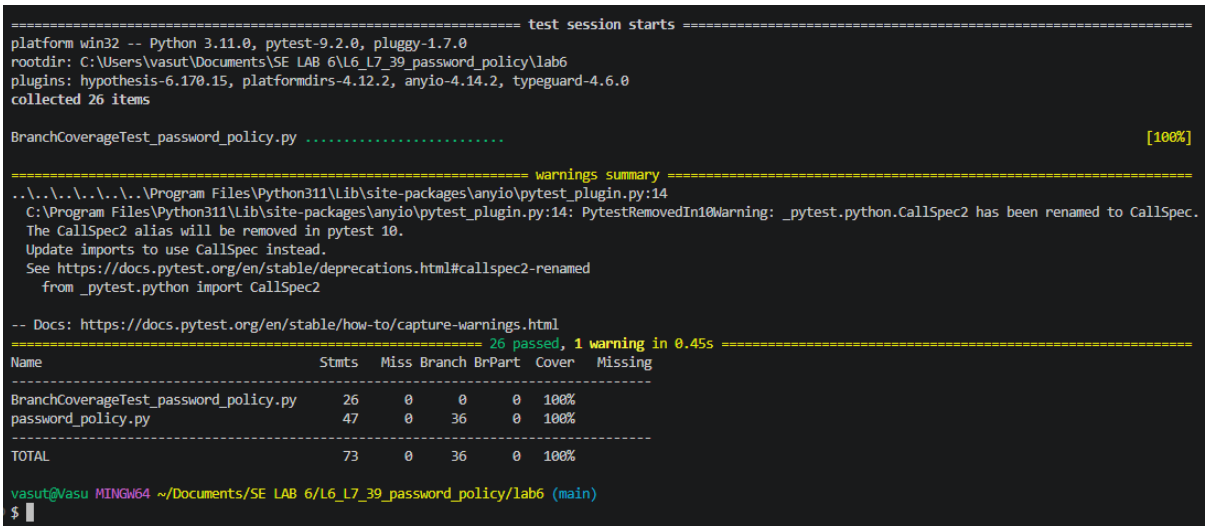


Figure 2: Branch coverage – 26 passed, password_policy.py 100% (36/36 branches, 0 partial)

Result: all 36 branches taken, 0 partial branches (BrPart = 0). Each decision was driven to both its True and False outcome, including boundary values:

- Length: 4 (too short), 8 and 64 (valid boundaries), 65 (too long).
- Empty string, so the character loop body never executes.
- One test per missing character class, plus a password with all classes present.
- strength: 13 vs 14 characters (good vs strong), 2 vs 3 issues (weak vs very weak).
- suggest_fix: an input ("too long") that falls through every if to the final return.

Line vs Branch Coverage

Line coverage only checks that each line runs at least once; branch coverage requires every decision to evaluate both True and False. A suite can reach 100% line coverage while still missing branches, for example by never testing a password of 8–64 characters where both length checks are False. Branch coverage is therefore the stricter measure and exposes untested logical paths that line coverage hides.

Summary

Test file	Tests	Line	Branch
test_password_policy.py (starter)	2	55%	52%
CodeCoverageTest_password_policy.py	15	100%	–
BranchCoverageTest_password_policy.py	26	100%	100%